



西安电子科技大学
XIDIAN UNIVERSITY

《操作系统》

第6讲 死锁

Dead Lock



人工智能学院

死锁



锁的实现

锁是一种**用于控制多个进程对共享资源访问的机制**，它的不同实现方式影响着系统的并发性能和死锁风险。

◆ 以中断启用与禁止 (interrupt enable/disable) 来实现锁

当一个进程**进入临界区**（访问共享资源的代码段）时，它可以通过**禁止中断**来实现锁的功能。因为在禁止中断的情况下，操作系统不会进行进程切换，其他进程也就无法抢占CPU并访问共享资源。**当进程完成对共享资源的访问后**，再启用中断。

◆ 以测试与设置 (test&set) 来实现锁

test&set是一种**原子操作**。它会测试一个变量（通常是锁变量）的值，如果该值为0（表示锁可用），就将其设置为1（表示锁已被占用）并返回 true；如果该值为1（表示锁已被占用），则直接返回 false。进程在进入临界区之前，**不断测试**这个锁变量，**只有当测试结果为true时才能进入临界区**。

◆ 以内存加载和存入 (load/store) 来实现锁

通过**内存加载和存入**的原子操作，也可以实现锁机制。**例如**，使用比较并交换 (Compare and Swap, CAS) 指令，该指令会比较内存中的值与期望值，如果相等，就将内存中的值更新为新值，并返回true；如果不相等，则不进行更新并返回false。

一、死锁定义

死锁(Deadlock)的定义



在多道程序中，由于多个并发进程共享系统的资源，如果使用不当可能会造成一种僵局，即当某个进程提出资源的使用请求后，使得系统中一些进程处于无休止的阻塞状态，在无外力的作用下，这些进程将无法继续下去，这就是死锁。

一、死锁定义

死锁(Deadlock)的定义

- 死锁发生在一组互相竞争或同步的进程之间
- 死锁是一种可能发生的现象
- 死锁是因竞争资源引发的冲突

Deadlock Concepts

- What is deadlock?

- Suppose a process holds resource A and requests resource B
 - at same time another process holds B and requests A
 - both are blocked and remain so
- Formal definition
 - *A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause*

- 什么是死锁?

- **假设**一个进程持有资源 A 并请求资源 B
 - 同时另一个进程持有 B 并请求 A
 - 两者都被阻塞且保持阻塞状态
- **正式定义**
 - 如果一组进程中的每个进程都在等待一个只有该组中另一个进程才能引发的事件, 那么这组进程就处于死锁状态

一、死锁定义

死锁——举例

办公室里有一台打印机和一台扫描仪。有两个程序正在运行：

- 程序A：它的任务是先扫描一份文件，然后把扫描的内容打印出来。
- 程序B：它的任务是先打印一份文件，然后把打印好的文件扫描存档。

死锁是如何一步步发生的？

第一步：同时开始

- 程序A运行了，它立刻锁定了扫描仪（因为它的第一步是扫描）
- 几乎在同一时刻，程序B也运行了，它立刻锁定了打印机（因为它的第一步是打印）

现在的情况是：

- A拿着扫描仪，等打印机
- B拿着打印机，等扫描仪

一、死锁定义

死锁——举例

死锁是如何一步步发生的？

第二步：互相等待

- 程序A扫描完成，它想去用打印机，但发现打印机被B锁定了。于是A说：“好吧，我等着B用完打印机。”
- 程序B打印完成，它想去用扫描仪，但发现扫描仪被A锁定了。于是B说：“好吧，我等着A用完扫描仪。”

第三步：无限等待（死锁）

- A在等B释放打印机
- B在等A释放扫描仪
- 两人都握着自己手里的资源不放，同时等着对方手里的资源。它们永远都等不到了，这就是死锁

二、死锁产生的环境和条件

死锁产生的环境

- ❖ 多道程序设计技术
- ❖ 多个并发进程
- ❖ 资源共享和独占
- ❖ 没有外力可以借助

死锁产生的必要条件(Coffman 1971年提出)

- ❖ 资源互斥使用(资源独占)
- ❖ 非剥夺控制(不可强占)
- ❖ 零散请求
- ❖ 循环等待

如果死锁发生，则这四个条件必然同时满足。

死锁的危害

- ❖ 轻则系统资源利用率严重下降，重则系统崩溃。

二、死锁产生的环境和条件

死锁 4 项必要条件

Four Conditions

- Mutual exclusion condition
 - Each resource is either currently assigned to exactly one process or is available.
- Hold and wait condition
 - process holding resources can request additional
- No preemption condition
 - previously granted resources cannot forcibly taken away
- Circular wait condition
 - must be a circular chain of 2 or more processes
 - each is waiting for resource held by next member of the chain

• 四项条件

◦ 互斥条件——资源有限

- 每个资源要么当前恰好分配给一个进程，要么处于可用状态

◦ 占有并等待条件——持有等待

- 持有资源的进程可以请求额外资源

◦ 不可剥夺条件——不能抢占

- 之前已分配的资源不能被强制剥夺

◦ 循环等待条件——循环等待

- 必须存在由两个或更多进程构成的循环链
- 每个进程都在等待链中下一成员所持有的资源

一、死锁定义

死锁——举例——打印机和扫描仪

死锁是如何一步步发生的？

必要条件	在这个例子中的体现
互斥	打印机和扫描仪都是“互斥资源”，一次只能给一个程序用。A用扫描仪时，B就不能用
占有并等待	A占着扫描仪，等待打印机。B占着打印机，等待扫描仪。完全符合
不可剥夺	操作系统不能强行把扫描仪从A手里抢过来给B用，也不能把打印机从B手里抢过来给A用。只能等程序自己释放
循环等待	A在等B（释放打印机），B在等A（释放扫描仪）。形成了一个“A->B->A”的循环等待圈

一、死锁定义

死锁推导公式

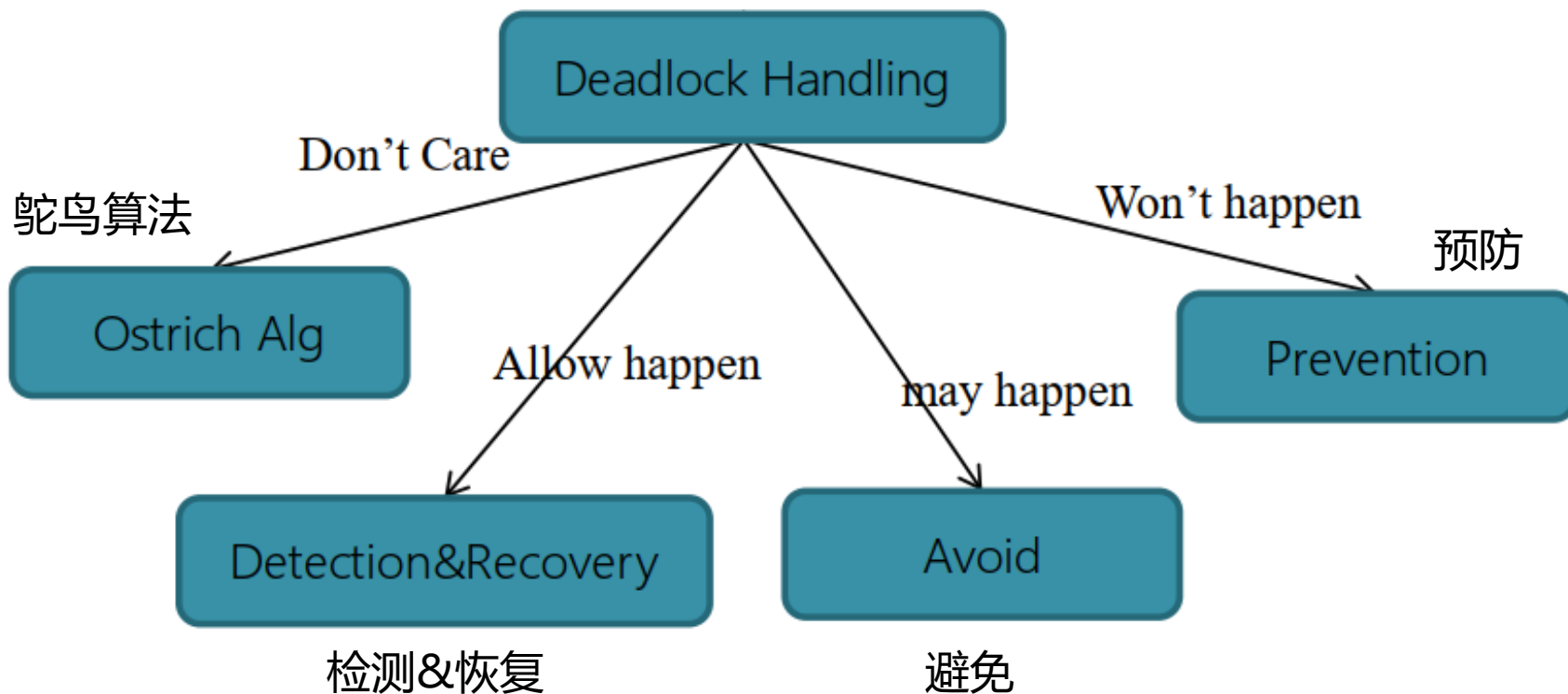
以单类型资源为例，推导系统**不发生死锁的条件**：

设系统有 m 个资源， n 个进程，每个进程最多需要 k 个资源。

若系统**不发生死锁**，**需满足**： $m \geq n(k-1) + 1$

推导：每个进程先分配 $k-1$ 个资源（共 $n(k-1)$ 个），此时系统至少剩 1 个资源，可让某一进程获取全部所需资源（ k 个）并完成，释放资源后其他进程也能依次完成，避免死锁。

三、死锁的应对策略



三、死锁的应对——不做预先处理

置之不理法——鸵鸟算法 (Ostrich Algorithm)



优点：简单，简化系统设计，节约成本。

缺点：安全性欠佳，适合于低可靠性和低安全性系统。

三、死锁的应对——适当容忍、坚决遏制

事后处理法——让死锁发生，事后处理

- ❖ 思想：可以容忍死锁的发生，事后处理。
- ❖ 优点：灵活，但不是所有情况都能容忍死锁发生的。

积极防御法——不让死锁发生

- ❖ 思想：以积极的遏制为出发点。
- ❖ 手段：
 - 死锁的预防：破坏死锁产生的条件，使其不具备发生的可能性。
 - 死锁的避免：允许存在发生死锁的可能性，但是在每一步进行资源分配时灵活处理，使得永远不能达到死锁状态。

四、死锁的预防

方法：破坏死锁产生的必要条件

❖ 破坏互斥条件

- 局限：“互斥”条件的破坏往往困难，而且对很多资源行不通。因此不是一种好的方案。

❖ 破坏不可剥夺条件

- 思想：允许一个进程还未执行完成时释放已经占有的资源(被剥夺使用权)。
- 局限：实现困难，为了恢复现场需要耗费很多时间和空间。因此只适合类似**CPU**、存储器这样的资源。

四、死锁的预防

❖ 破坏零散请求条件

- 思想：进程创建时就由系统分配了所有需要的资源，然后才执行，并且以后没有资源申请要求，进程执行完后，释放资源。
- 局限：系统效率低，资源浪费严重，并发性下降。

❖ 破坏循环等待条件

- 方法：给资源编号，进程必须按序申请资源。
- 局限：资源编号困难；资源的编号很难和进程申请资源的顺序一致。

结论

- ❖ 死锁的预防是以破坏死锁产生的必要条件为基本方法，从而防止死锁发生的。由于对资源的申请加上了诸多的限制，因此这种策略虽有一定的效果，但其资源的利用率和效率比较低，很难令人满意。

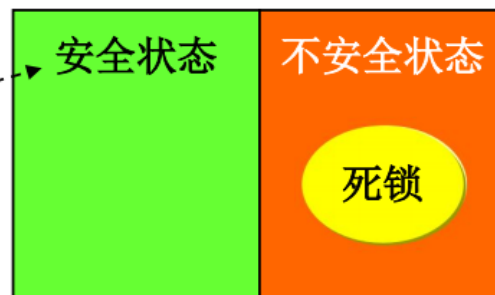
五、死锁的避免

思想 (动态地判断资源的使用情况和系统的状态)

- ❖ 允许死锁产生的条件存在(可能会发生死锁)，但通过动态的、明智的选择——在分配资源之前，系统判断假若满足进程的要求是否会发生死锁，如果会，资源就不予分配，从而确保永远不会到达死锁点，避免死锁的发生。
- ❖ 优点：比预防策略更为灵活实用，允许更多的并发，其资源利用率和效率也更高。

系统的状态

安全状态：指在某个时刻，当多个进程动态的申请资源时，如果存在一种顺序，使得系统按照这种顺序逐次地为每个进程分配所需资源后，每个进程都可以在最终得到最大需求量后，依次顺利地完成。

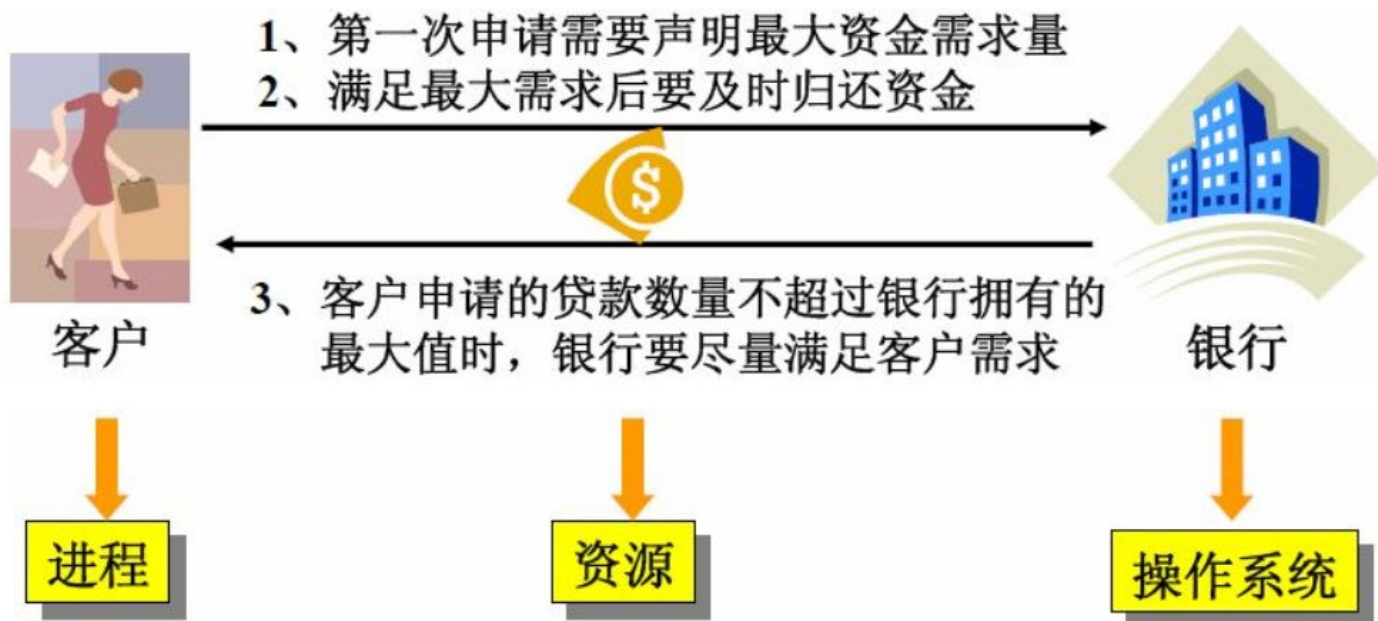


避免死锁的**关键**就是：让系统在动态分配资源的过程中，不要进入不安全状态。

五、死锁的避免

单银行家算法(Banker's Algorithm) (资源分配时避免死锁)

- ❖ 1965年由Dijkstra设计。
- ❖ 基本思想：借用了银行借贷系统的分配策略。基于这样一些规则：



五、死锁的避免

- ❖ 举例：假设一个银行拥有资金数量为10（单位省略），现在有4个客户a, b, c, d要来贷款，所需最大资金需求量为8, 5, 6, 7，为方便期间我们用图形表示如下。

客户	已用资金	最大需求	仍需资金
a	0	8	8
b	0	5	5
c	0	6	6
d	0	7	7
银行剩余资金		10	

初始状态

客户	已用资金	最大需求	仍需资金
a	1	8	7
b	2	5	3
c	1	6	5
d	2	7	5
银行剩余资金		4	

状态1

Q：如何分配才能保证银行资金能够顺利周转？

五、死锁的避免

客户	已用资金	最大需求	仍需资金
a	1	8	7
b	5	5	0
c	1	6	5
d	2	7	5
银行剩余资金		1	

状态2

客户	已用资金	最大需求	仍需资金
a	1	8	7
b	—	—	—
c	1	6	5
d	2	7	5
银行剩余资金		6	

状态3

客户	已用资金	最大需求	仍需资金
a	1	8	7
b	—	—	—
c	6	6	0
d	2	7	5
银行剩余资金		1	

状态4

客户	已用资金	最大需求	仍需资金
a	1	8	7
b	—	—	—
c	—	—	—
d	2	7	5
银行剩余资金		7	

状态5

五、死锁的避免

客户	已用资金	最大需求	仍需资金
a	8	8	0
b	—	—	—
c	—	—	—
d	2	7	5
银行剩余资金		0	

状态6

客户	已用资金	最大需求	仍需资金
a	—	—	—
b	—	—	—
c	—	—	—
d	2	7	5
银行剩余资金		8	

状态7

客户	已用资金	最大需求	仍需资金
a	—	—	—
b	—	—	—
c	—	—	—
d	7	7	0
银行剩余资金		3	

状态8

客户	已用资金	最大需求	仍需资金
a	—	—	—
b	—	—	—
c	—	—	—
d	—	—	—
银行剩余资金		10	

状态9

$b \rightarrow c \rightarrow a \rightarrow d$

五、死锁的避免

死锁避免的资源分配公式——银行家算法

可用资源向量 *Available*: 系统中各类型资源的可用数量

最大需求矩阵 *Max*: 每个进程对各类型资源的最大需求量

已分配矩阵 *Allocation*: 每个进程已获得的各类型资源数量

需求矩阵 *Need*: 每个进程还需要的各类型资源数量, 公式为:

$$Need[i][j] = Max[i][j] - Allocation[i][j]$$

其中 i 表示进程编号, j 表示资源类型。

□ 安全性检查流程

假设系统有 n 个进程, m 种资源, 判断当前状态是否安全的步骤 (伪代码):

1. 找到一个进程 i , 使得 $Need[i] \leq Available$ (进程 i 可以获取所需资源并完成)
2. 假设进程 i 完成后释放资源, 更新 $Available = Available + Allocation[i]$
3. 重复步骤 1-2, 直到所有进程都能完成 (安全状态), 或无法找到满足条件的进程 (不安全状态, 可能死锁)

五、死锁的避免

多项资源银行家算法*

- ❖ 适用于一个进程申请多个资源的情况。
- ❖ 举例：系统中有以下资源：5台打印机，7个手写板，8台扫描仪，9个读卡器，共有5个进程T1、T2、T3、T4、T5共享这些资源。各进程所需最大资源量和当前各进程已经得到的资源数量如下图，问如果进程T2此时希望得到1台打印机,1个手写板,2个读卡器是否可以满足？

进程	R1	R2	R3	R4
T1	2	4	3	1
T2	2	2	0	5
T3	1	5	5	0
T4	5	0	1	3
T5	0	3	3	3
合计	10	14	12	12

各进程所需最大资源量

进程	R1	R2	R3	R4
T1	0	1	2	1
T2	0	0	0	0
T3	0	3	4	0
T4	2	0	0	1
T5	0	0	1	3

当前各进程已分配资源

进程	R1	R2	R3	R4
T1	0	1	2	1
T2	1	1	0	2
T3	0	3	4	0
T4	2	0	0	1
T5	0	0	1	3
合计	3	5	7	7

假设分配状态

为方便讨论，我们用向量来表示资源分配及占用情况：

1、sum向量：表示系统资源总量

$$\text{sum} = (5, 7, 8, 9)$$

2、allocation向量：表示当前系统已分配资源

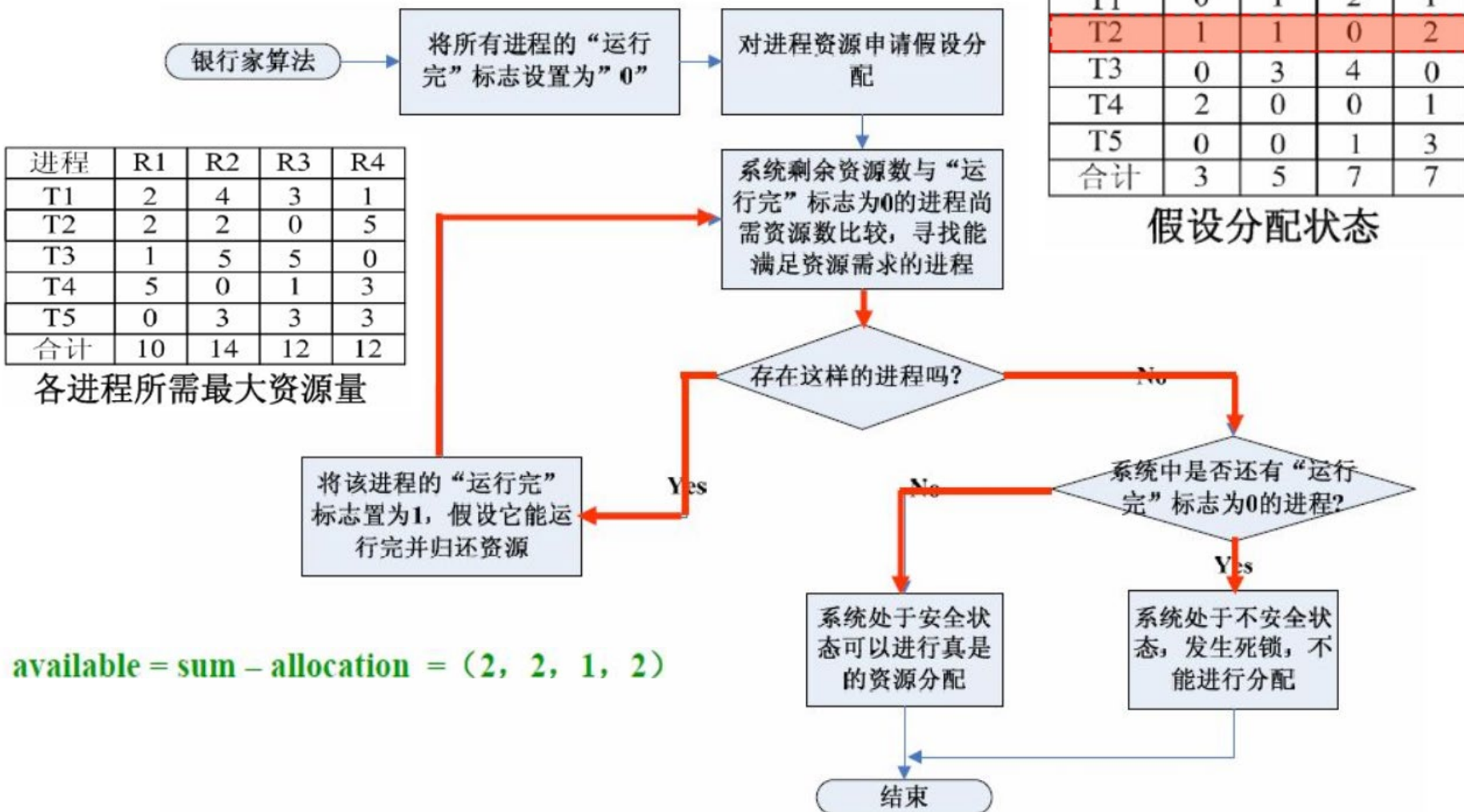
$$\text{allocation} = (3, 5, 7, 7)$$

3、available向量：表示系统剩余资源

$$\text{available} = \text{sum} - \text{allocation} = (2, 2, 1, 2)$$

五、死锁的避免

❖ 算法流程



五、死锁的避免

单银行家算法和多银行家算法的异同

❖ 不同点:

- 多银行家算法使用向量进行计算，单银行家算法使用单个整数值。

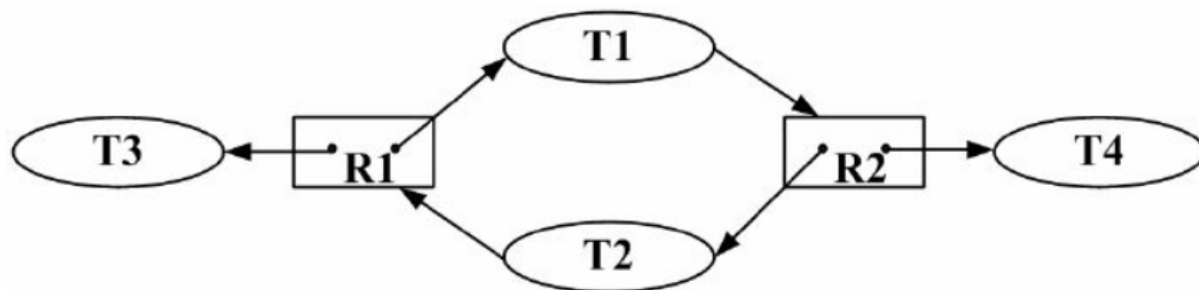
❖ 相同点

- 思想和算法流程完全一样。

六、死锁的检测和解除

死锁的检测*

- ❖ 检测工具——资源分配图
- ❖ 资源分配图：是描述进程申请资源和资源分配情况的关系模型图。表示系统中某个时刻进程对资源的申请和占有情况。

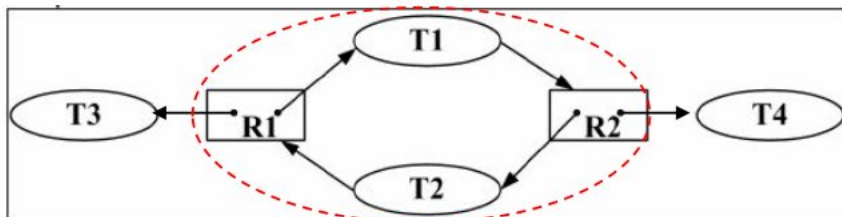


规则：

- (1) 圆(椭圆)表示一个进程；
- (2) 方块表示一个资源类，其中的圆点表示该类型资源中的单个资源；
- (3) 从资源指向进程的箭头表示资源被分配给了这个进程；
- (4) 从进程指向资源的箭头表示进程申请一个这类资源；

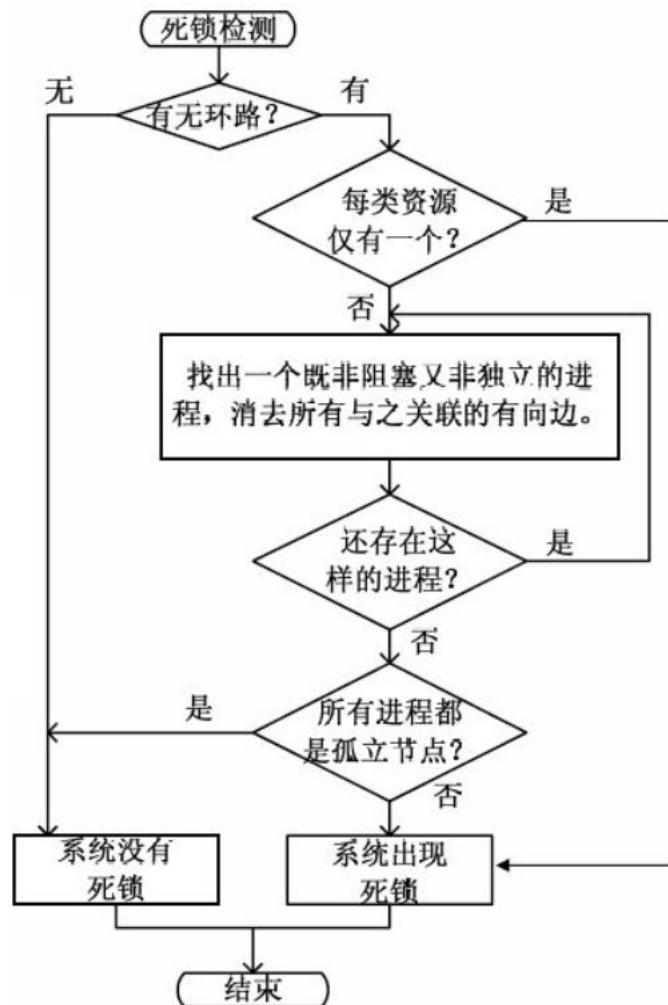
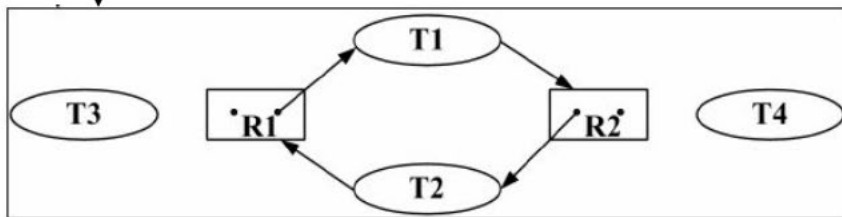
六、死锁的检测和解除

死锁检测的流程



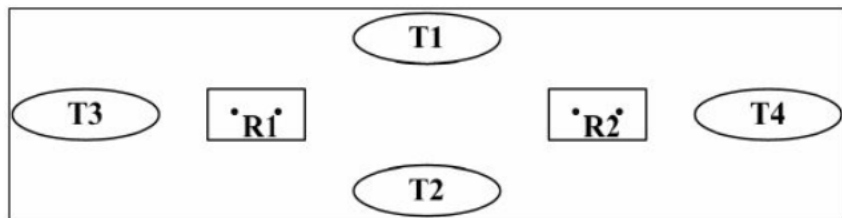
- P_i : 第 i 个进程; R_j : 第 j 个资源
- $|(P_i, R_j)|$: 进程 P_i 申请 R_j 的数量
- $|(R_j, P_k)|$: 资源 R_j 已分配给 P_k 的数量
- W_j : R_j 类资源的总个数

资源分配需满足约束: $|(P_i, R_j)| + \sum_k |(R_j, P_k)| \leq W_j$



六、死锁的检测和解除

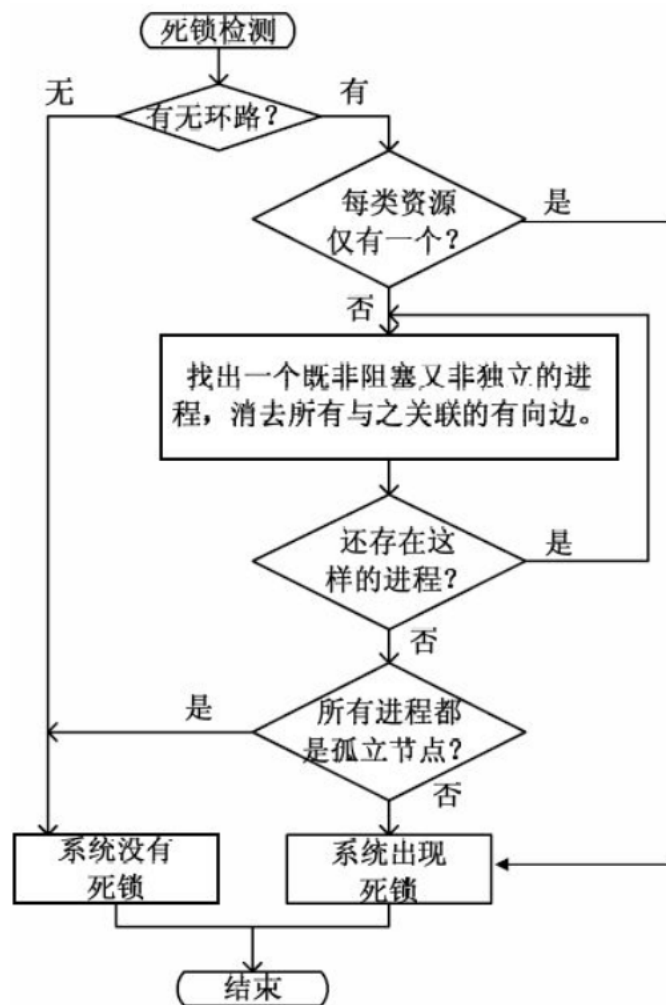
死锁检测的流程



资源分配图中的所有进程如果都能化简成孤立结点，则这个资源图就是**可完全化简的** (completely reducible)；反之，就是**不可完全化简的** (irreducible)。

死锁定理：如果一个系统状态为死锁状态，**当且仅当**资源分配图是不可完全化简。也即，如果资源图中所有的进程都成为孤立结点，则系统不会死锁；否则系统状态为死锁状态。

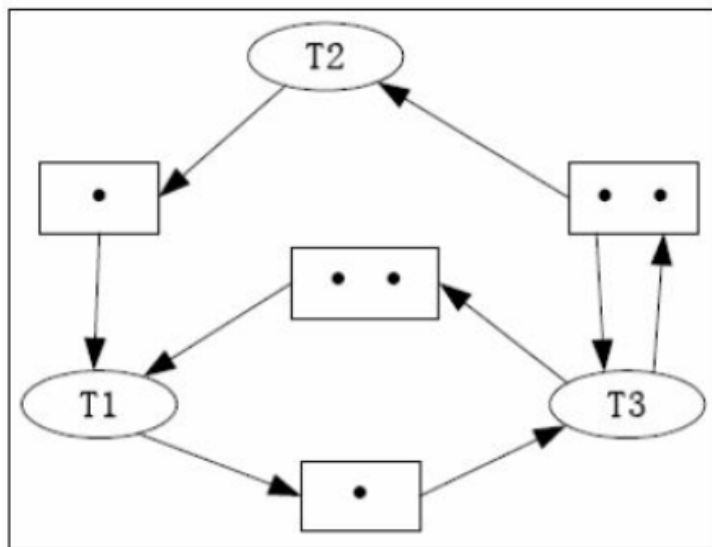
结论：系统不会发生死锁



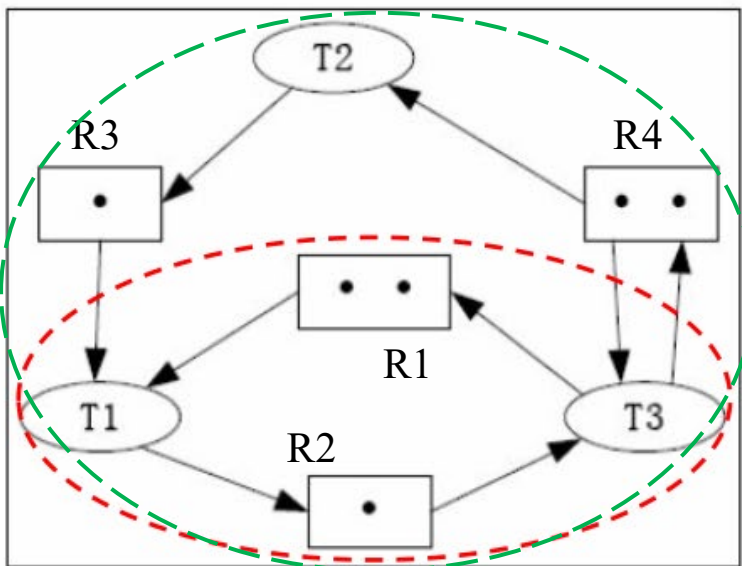
六、死锁的检测和解除

举例

❖ 资源分配图如下，请分别化简并说明是否会发生死锁。



六、死锁的检测和解除



Step1.检测有无环路



Setp2.检测环路中每个资源类中是否只有一个资源

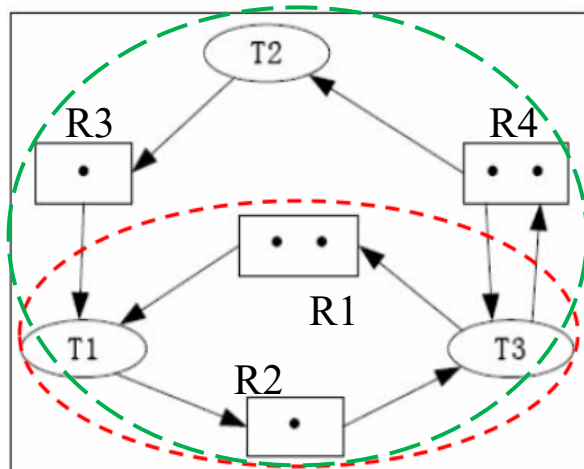


Step3.在环路中查找非阻塞也非独立的进程



结论：此资源分配图无法化简，因此必定死锁。

六、死锁的检测和解除



T1 → R2 (T1请求R2)
T3 → R1 (T3请求R1)
T3 → R4 (T3请求R4)
T2 → R3 (T2请求R3)

R1 → T1 (R1被T1占用)
R2 → T3 (R2被T3占用)
R3 → T1 (R3被T1占用)
R4 → T2 (R4被T2占用)
R4 → T3 (R4被T3占用)

检测流程

➤ Step1: 环路检测

- **环路1** (直接互斥) : T1 → R2 → T3 → R1 → T1 (T1 和 T3 互相等待: T1 等 R2, T3 等 R1)
- **环路2** (扩展环路) : T1 → R2 → T3 → R4 → T2 → R3 → T1 (T1 等 R2, T3 等 R4, T2 等 R3, R3 被 T1 占用)

➤ Step2: 环路中每个资源类是否只有一个资源

- **环路1** (T1-T3) 涉及资源: R1 和 R2
R1: 有两个资源 (多实例) ; R2: 有一个资源 (单实例)
- **环路2** (T1-T3-T2) 涉及资源: R2、R4、R3
R2: 单实例; R4: 有两个资源 (多实例) ; R3: 单实例

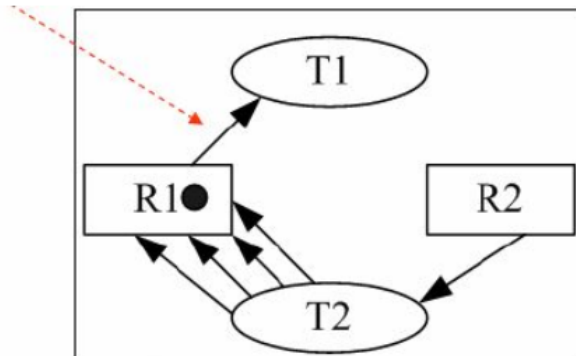
➤ Step 3. 在环路中查找非阻塞且非独立的进程 (尝试化简图)

- **T1**: 占用 R1 (的一个实例) 和 R3 (单实例), 但请求 R2 (R2 被 T3 占用, 且 R2 无空闲) → **阻塞, 不可化简**
- **T2**: 占用 R4 (的一个实例), 但请求 R3 (R3 被 T1 占用, 且 R3 无空闲) → **阻塞, 不可化简**
- **T3**: 占用 R2 (单实例) 和 R4, 请求 R1 和 R4 (R4 无空闲) → **阻塞, 不可化简**

六、死锁的检测和解除

临时资源的死锁检测

- ❖ 临时性资源：即可消耗的资源。如信号、消息、邮件等。
- ❖ 特点：没有固定数目；不需要释放。
- ❖ 表述方式——重定义的资源分配图
 - 规则：
 - ① 圆表示一个进程；
 - ② 方块表示一个资源类，其中的圆点表示该类型资源中的单个资源；
 - ③ 由进程指向资源的箭头表示该进程申请这种资源，一个箭头只表示申请一个资源；
 - ④ 由资源类指向进程的箭头表示该进程产生这种资源，一个箭头可表示产生一到多个资源，每个资源类至少有一个生产者进程。



六、死锁的检测和解除

❖ 死锁判断标准

➤ 分析：

- ① 对于临时性资源来讲，它有生产者，生产者会源源不断的生产资源，因此只要生产者进程不被阻塞，可以认为资源最终一定是充分的，可以满足各消费进程的需要。
- ② 一个进程死锁意味着永久被阻塞。需要的资源可以满足则进程一定不会死锁。

➤ 结论：

判断系统是否死锁的关键在于判断生产者进程的状态，若生产者进程不被阻塞，则可以认为它总会生产出该类资源，也就是说，申请这类资源的所有申请者进程都可以得到满足。

六、死锁的检测和解除

❖ 检测方法——化简

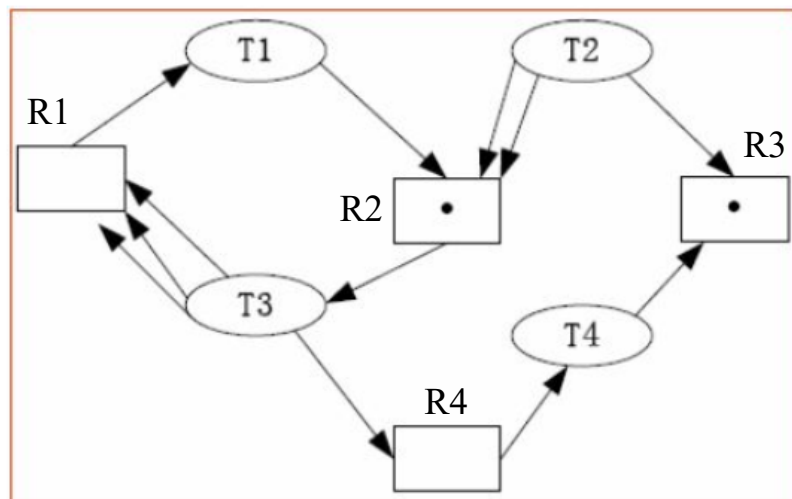
➤ 方法:

- ① 从那些没有阻塞的进程入手，删除那些没有阻塞的进程的请求边，并使资源类中资源数（图中黑点的数目）减1。
- ② 重复以上步骤直至：
 - a. 图中所有的请求边都已经删除，则不会死锁
 - b. 图中仍然存在请求边但无法再化简，系统死锁。

六、死锁的检测和解除

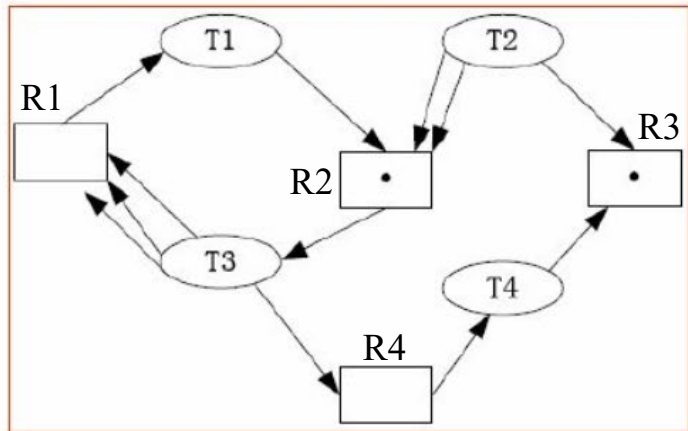
死锁检测举例

- ❖ 资源分配图如下（表示临时性资源），请分别化简并说明是否会发生死锁：

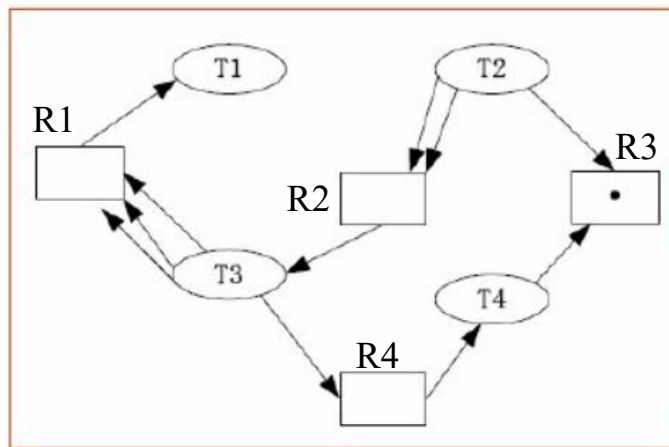


(b)

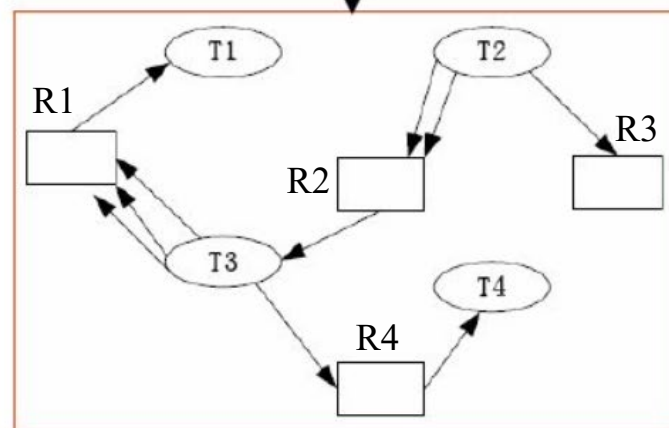
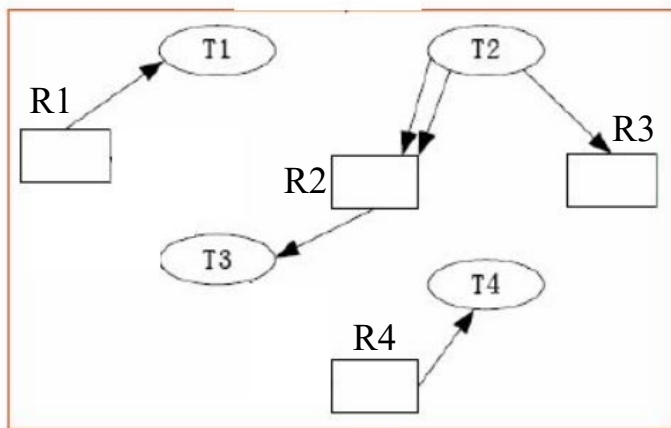
六、死锁的检测和解除



查找非阻塞进程



结论：会发生死锁



六、死锁的检测和解除

死锁的解除

❖ 重新启动

- 这是一种常用但比较粗暴的方法，虽然实现简单，但会使之前的工作全部白费，造成很大的损失和浪费。

❖ 撤消进程

- 死锁发生时，系统撤消造成死锁的进程，解除死锁。
- 一次性撤消所有的死锁进程。损失较大。
- 逐个撤消，分别收回资源。具体做法：系统可以先撤消那些优先级低的、已占有资源少或已运行时间短的、还需运行时间较长的进程，尽量减少系统的损失。

六、死锁的检测和解除

❖ 剥夺资源

- 死锁时，系统保留死锁进程，只剥夺死锁进程占有的资源，直到解除死锁。选择被剥夺资源进程的方法和选择被撤消进程相同。

❖ 进程回退

- 死锁时，系统可以根据保留的历史信息，让死锁的进程从当前状态向后退回到某种状态，直到死锁解除。
- 实现方法：可以通过结合检查点或回退（**checkpoint/Rollback**）机制实现。进程某一时刻的瞬间状态叫做检查点，可以定期设置检查点。系统保存所有的检查点。一旦系统检查到有某个进程卷入了死锁，该进程就会被终止，剥夺它占有的所有资源。然后，系统查看保存的检查点信息，重新建立该进程的状态，从上次检查点的位置重新执行。目前发展已比较成熟，广泛用于**DBMS**中。

作业-选做

思考二选一：

1. 死锁、活锁、饥饿、循环等待之间的联系区别
2. 采用银行家算法、资源分配图法分析死锁的可能性